

Diseño y Análisis de Algoritmos

Prueba Formativa 1

Profesor: Felipe Ruiz

Ayudante: Cristian Nettle

Nombre: _____

Fecha: _____

Indicaciones

- Duración: 90 minutos.
- Puntaje total: 90 puntos.
- Justifique sus respuestas cuando se solicite. La corrección y la claridad del razonamiento son parte de la nota.
- Pseudocódigo claro y *self-contained* (es decir: no usar librerías externas).
- Trabajo individual. No se permite el uso de apuntes, calculadoras ni dispositivos electrónicos.
- Encierre en un recuadro la **respuesta final** (complejidad, valor óptimo, etc.). Rotule claramente: (a), (b), etc.
- Durante la prueba, solo se responden dudas de **enunciado** (no de estrategias).
- Se otorgan puntos por **ideas correctas** aunque existan errores menores de aritmética o notación. Respuestas ambiguas no serán consideradas.
- Respuestas ilegibles, fuera de orden o sin etiquetas pueden no ser corregidas.
- Prohibida toda comunicación con terceros o uso de herramientas de IA. Infracciones serán informadas según reglamento.

1. Torres de Hanoi**(30 puntos)**

En el **rompecabezas de Hanoi**, ideado por Édouard Lucas a fines del s. XIX, hay tres postes: *origen*, *apoyo* y *destino*. En el *origen* hay n discos (del más grande abajo al más pequeño arriba). En cada paso solo puedes mover **un** disco y **nunca** colocar uno grande sobre uno más pequeño. El objetivo es trasladar toda la pila al *destino* utilizando el *apoyo*.

El siguiente es el *pseudocódigo* del algoritmo recursivo que efectúa los movimientos:

Algoritmo 1 HANOI(n , origen, apoyo, destino)

```
1  if  $n = 1$ 
2      MOVER(1, origen, destino)            $\triangleright$  Mover( $n$ , origen, destino) es  $O(1)$ 
3      return
4  HANOI( $n - 1$ , origen, destino, apoyo)
5  MOVER( $n$ , origen, destino)
6  HANOI( $n - 1$ , apoyo, origen, destino)
```

- (a) Identifique la **ecuación de recurrencia** $T(n)$ que modela el *número mínimo de movimientos*, e indique condiciones base. **(10 puntos)**
- (b) Resuelva la recurrencia *por sustitución* (expansión) hasta obtener la *forma cerrada* y exprese además la complejidad temporal en notación $\mathcal{O}(\cdot)$. Explique brevemente por qué su solución es correcta. **(20 puntos)**

2. D&C diferencia máxima**(30 puntos)**

Escriba un algoritmo de dividir y conquistar que encuentre la diferencia máxima entre cualesquiera dos elementos de un arreglo dado de n números (no necesariamente distintos) en tiempo $O(n)$. Por ejemplo, con entrada $A = [4, 5; 10; -2; \pi; -7, 115]$, su algoritmo debe devolver 17,115.
(30 puntos)

Nota: Para el puntaje completo, su respuesta debe hacer uso del método de dividir y conquistar. Se otorgará puntaje parcial por un método de dividir y conquistar de $O(n \log n)$.

3. Subir escaleras en Valparaíso**(30 puntos)**

Durante un paseo por **Valparaíso**, decides subir una de sus escaleras icónicas en Cerro Alegre. La escalera tiene n peldaños y en cada paso puedes subir **1** o **2** peldaños. Tu objetivo es contar **de cuántas formas distintas** puedes llegar exactamente al peldaño n desde el suelo (peldaño 0).

Nota: Dos formas son distintas si en alguna posición del recorrido subes 2 peldaños en lugar de 1 peldaño (o viceversa).

- (a) Diseñe un algoritmo que, dado n , calcule el número de formas de llegar al peldaño n en $O(n)$ tiempo y $O(n)$ espacio adicional. Escriba el pseudocódigo de una función que reciba n y retorne dicho número, y justifique en 2–3 líneas por qué se cumplen esas cotas. **(25 puntos)**
- (b) Ahora suponga que existe (y se recibe por parámetro) un peldaño r que está roto (no es posible caer en r). Describa en palabras cómo cambiaría el algoritmo de **(a)** para manejar este caso. **(5 puntos)**